# Point Operations & Histograms

Week 2 — Session 1: Reading the histogram & per-pixel fixes

Practical Image Processing

Lecture 2.1 ▪ 60–70 min

# Recap & today

- Wk1: an image is numbers on a grid; watch BGR, uint8, and `[y,x]` indexing.
- Today (Session 1): the **histogram** — the tool that tells you what a transform should do — and **point operations** that transform each pixel by itself.
- Session 2 will build on this for **thresholding** and advanced enhancement.

**Why it matters**

DocScan's meter photos are often too dark or low-contrast. Fixing exposure is step one to reading the digits.

# Roadmap

The histogram

Point operations

Contrast enhancement

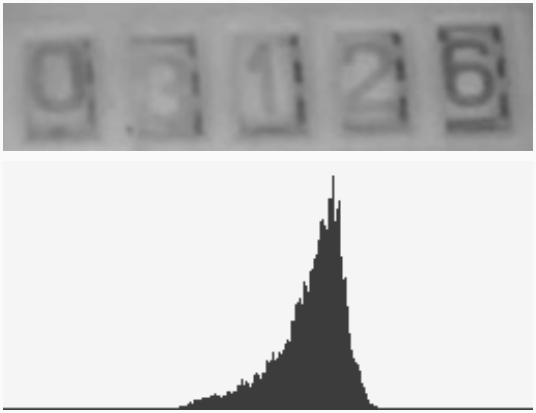
In-class exercise

Wrap-up

# The histogram

# The histogram: a census of brightness

A real meter crop and its histogram:

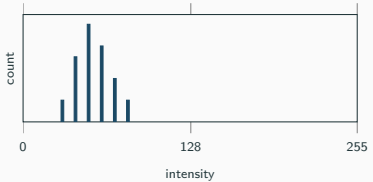


- $x$-axis: intensity 0–255. $y$-axis: how many pixels have it.
- This histogram is bunched $\rightarrow$ **low contrast** (digits barely stand out).
- Read the histogram *before* touching the image.

**Definition**

For a gray image, $h(k)$ = number of pixels with intensity $k$, for $k = 0, \ldots, 255$.

# Reading a histogram




- Peak on the **left** = dark (under-exposed). On the **right** = bright.
- **Narrow** = low contrast. **Spread across the range** = good contrast.
- Two peaks (**bimodal**) = object vs background — a gift for thresholding (Session 2).

# Four histogram shapes you must recognise




**Key idea**

Diagnose the shape first — it decides *whether* and *which* enhancement to apply.

# Computing a histogram

```python
import cv2, numpy as np
gray = cv2.cvtColor(cv2.imread('E:/data/abfa_image/numerators/num_3494.jpg'),
                    cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])  # 256 bins
print(gray.min(), gray.max(), gray.mean())   # quick exposure check
```

**Key idea**

The histogram throws away *where* pixels are and keeps *how many* of each value. That's exactly what you need to reason about exposure and contrast.

# Worked example: histogram of a tiny image

A $4 \times 4$ patch of intensities:

| | | | |
|---|---|---|---|
| 10 | 10 | 40 | 40 |
| 10 | 20 | 40 | 60 |
| 20 | 20 | 60 | 60 |

20,40,60,90

Tally each value:

| value $k$ | 10 | 20 | 40 | 60 | 90 |
|---|---|---|---|---|---|
| count $h(k)$ | 3 | 4 | 4 | 4 | 1 |

- 16 pixels total; range is only $[10, 90]$ — nothing near white.
- Bunched & dark $\rightarrow$ a stretch or gamma will help.

# Point operations

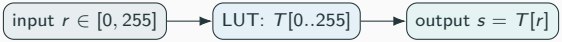
# Point operations: one pixel at a time

- A **point operation** maps each pixel's value through a function $s = T(r)$ — independent of neighbours.
- Brightness: $s = r + b$. Contrast: $s = a \cdot r$. Negative: $s = 255 - r$.
- Implemented as a **lookup table** (LUT): compute $T$ once for 0–255, apply to all pixels fast.

**Math**

General linear: $s = a \cdot r + b$ — $a$ scales contrast, $b$ shifts brightness. Then clip to 0–255.

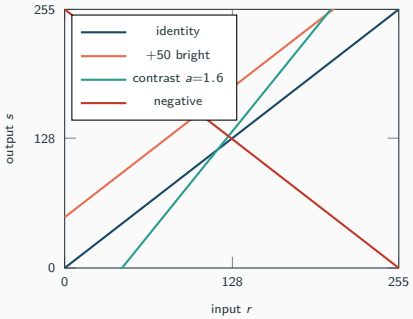
# The LUT: compute once, apply everywhere




256-entry array — one table lookup per pixel,
no arithmetic in the inner loop.

- Any per-pixel function (linear, gamma, negative, threshold) is *just a different table*.
- Cost is independent of image size in the table build; application is one pass, $O(N)$.

# The transfer function $s = T(r)$




- The **shape** of $T$ is the whole story.
- Slope = contrast; vertical offset = brightness.
- Anything above 255 or below 0 is **clipped** (flat ends of the real curve).

# Worked example: a linear map with clipping

**Math**

$s = \mathrm{clip}(a\,r + b,\ 0,\ 255)$ with $a = 1.5,\ \ b = -40$.

| $r$ | 20 | 60 | 120 | 200 | 240 |
|---|---|---|---|---|---|
| $ar + b$ | $-10$ | 50 | 140 | 260 | 320 |
| $s$ (clipped) | **0** | 50 | 140 | **255** | **255** |

- Dark pixels ($r = 20$) get crushed to 0; bright ones ($\geq 200$) saturate at 255.
- Contrast $a > 1$ steepens the middle but **loses** detail at both ends.

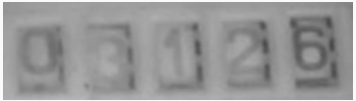
**✕ Pitfall**

Forgetting to clip (and letting uint8 wrap around) turns bright pixels black — a classic bug.

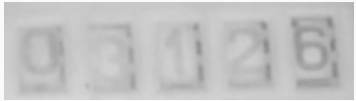
# Gamma correction — on the real meter



$\gamma < 1$ (darker)



original

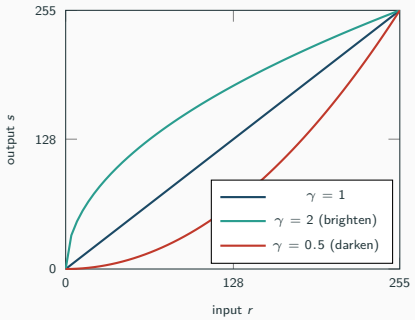


$\gamma > 1$ (brighter)

**Math**

$s = 255\,(r/255)^{1/\gamma}$ — a *nonlinear* curve that brightens shadows without blowing out highlights.

# The gamma curve, drawn




- Curve **above** the diagonal lifts shadows (brightens); **below** darkens.
- Endpoints stay pinned at 0 and 255 — no clipping, unlike linear.
- The steep part near dark values is why gamma recovers hidden digits.

# Brightness/contrast vs gamma

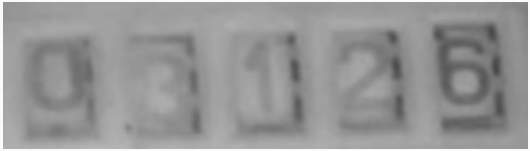
- Linear ($ar + b$) shifts/scales everything equally — can clip highlights or crush shadows.
- Gamma is **nonlinear**: it lifts dark tones more than bright ones (or vice versa).
- For a dark meter photo, gamma often recovers digits that a linear brighten would wash out.

**✔ Good practice**

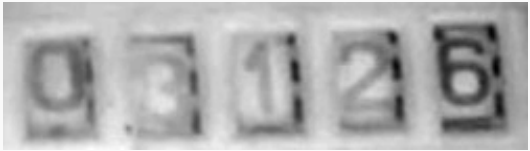
Match the tool to the histogram: shifted-but-spread → brightness; bunched → contrast/gamma.

# Contrast enhancement

# Contrast stretching (min–max)



before



after: stretched to 0–255

- Map the actual $[\min, \max]$ to the full $[0, 255]$.
- Simple, reversible, and it can't invent detail — just uses the full range.

**Math**

$$s = 255 \frac{r - \min}{\max - \min}$$

# Worked example: stretching the tiny patch

**Math**

Patch range was $[\min, \max] = [10, 90]$, so $s = 255\dfrac{r - 10}{80}$.

| $r$ | 10 | 20 | 40 | 60 | 90 |
|---|---|---|---|---|---|
| $s$ | 0 | 32 | 96 | 159 | 255 |

- The darkest value goes to pure black, the brightest to pure white.
- Same *ordering*, now spanning the full range — contrast is maximised without new detail.

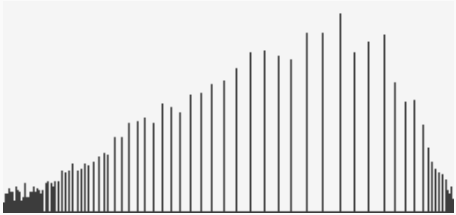
**✕ Pitfall**

A single bright speck (outlier max) can ruin a min–max stretch. Use **percentile** clipping (e.g. 2%–98%) instead.

# Histogram equalization



equalized meter



its histogram — spread out

- Redistributes intensities so the histogram is (roughly) **uniform**.
- Uses the **cumulative histogram** (CDF) as the mapping function.
- Great for revealing detail in low-contrast images — but can over-amplify noise.

# Equalization = use the CDF as the map

**Math**

$s = \mathrm{round}(255 \cdot \mathrm{CDF}(r))$, where $\mathrm{CDF}(r) = \dfrac{1}{N} \sum_{k \le r} h(k)$.

- The CDF is monotonic and rises fastest where pixels are *crowded*.
- Crowded values get spread apart (more contrast there); empty ranges get compressed.
- Result: the output histogram is as flat as a discrete image allows.

# Worked example: equalizing the tiny patch

| $r$ | 10 | 20 | 40 | 60 | 90 |
|---|---|---|---|---|---|
| $h(k)$ | 3 | 4 | 4 | 4 | 1 |
| cumulative | 3 | 7 | 11 | 15 | 16 |
| $\mathrm{CDF} = \mathrm{cum}/16$ | 0.19 | 0.44 | 0.69 | 0.94 | 1.00 |
| $s = 255\,\mathrm{CDF}$ | 48 | 112 | 175 | 239 | 255 |

- Five bunched dark values are pushed across the whole range.
- Notice the spacing follows the counts — dense region (10–60) fans out most.

# The three, in code

```
stretch = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)
equalized = cv2.equalizeHist(gray)
lut = np.array([((i/255)**(1/0.5))*255 for i in range(256)], np.uint8)
gamma = cv2.LUT(gray, lut)                  # gamma = 0.5
```

**Note**

`cv2.LUT` applies a 256-entry table to every pixel in one fast pass — the pattern for any point op.

# In-class exercise

# In-class exercise (10 min)

**Diagnose from the histogram**

You are handed three meter photos. Their histograms are:

1. a tall spike near 0, nothing above 80;
2. two separated humps, one near 40 and one near 210;
3. a smooth spread from 0 to 255.

For each: is it under/over-exposed, low-contrast, or fine? Which operation would you apply, and which needs *no* enhancement? Which one is easiest to threshold, and why?

# Debrief

- (1) Under-exposed & low-contrast $\rightarrow$ gamma ($\gamma > 1$) or stretch; digits hide in the dark.
- (2) Bimodal $\rightarrow$ already separable; *ideal* for thresholding (object vs background).
- (3) Well-exposed, full range $\rightarrow$ leave it; enhancing may only amplify noise.

**Key idea**

The histogram tells you *whether* and *which* enhancement — before you try anything.

# Wrap-up

# Pitfalls & good practice

**✗ Pitfall**

- Not clipping → uint8 wrap-around (bright becomes black).
- Min–max stretch wrecked by one outlier pixel.
- Equalizing before you know the histogram shape.

**✓ Good practice**

- Print `min`/`max`/`mean` before choosing a fix.
- Prefer **deterministic** ops (gamma/stretch) in a pipeline.
- Build the LUT once; apply with `cv2.LUT`.

# What you should now be able to do

- Read a histogram to diagnose exposure and contrast.
- Apply brightness, contrast, gamma, stretching, and equalization — and know which fits.
- Reason about a point operation as a transfer function $s = T(r)$ and a LUT.
- Hand-compute a stretch and an equalization on a small histogram.

# Next session

**Session 2 — Thresholding & advanced enhancement**

- Turn a gray image into a clean digit **mask**: global (Otsu) vs **adaptive** thresholding.
- **CLAHE**, histogram matching, and enhancing **color** the right way (value channel).
- The full DocScan chain: enhance $\rightarrow$ threshold $\rightarrow$ clean.

Bring your histogram diagnoses from today — we act on them next time.

## Look at the histogram first.

**It tells you what the image needs before you change a single pixel.**